

Internet of Things (IoT)  
/  
Internet des Objets (IdO)  
/  
Application Protocols : HTTP

Gilles Menez  
University Nice Côte d'Azur  
email : menez@unice.fr  
www : [www.i3s.unice.fr/~menez](http://www.i3s.unice.fr/~menez)

April 8, 2026: V 1.0

## What you should learn ...

1. We show some configurations of HTTP as application protocol between ESP and ...

## Two remarks

1. When the code is given ... YOU ARE SUPPOSED TO study IT !  
and not only to make it run ... I already KNOW it is running ;-)
2. Sections titled "TODO : Votre travail !" explain what you have to experiment or create by yourself from examples.

Do not hesitate to be proactive and to propose improvements of questions/answers/requirements !

---

# Table of Contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>"Faire" du HTTP en ligne</b>                                 | <b>5</b>  |
| 1.1      | cURL . . . . .                                                  | 5         |
| 1.2      | First request . . . . .                                         | 6         |
| 1.3      | "httpbin.org" : Test Server . . . . .                           | 7         |
| 1.4      | See more of "request / response" exchange . . . . .             | 8         |
| 1.5      | Few IMPORTANT examples . . . . .                                | 9         |
| 1.5.1    | "GET request" WITH parameters . . . . .                         | 9         |
| 1.5.2    | "GET request" WITH parameters AND headers . . . . .             | 10        |
| 1.5.3    | "POST request" WITHOUT parameter . . . . .                      | 11        |
| 1.5.4    | "POST request" WITH parameters AND JSON body . . . . .          | 12        |
| 1.6      | TODO : cURL . . . . .                                           | 13        |
| <b>2</b> | <b>HTTP as application protocol</b>                             | <b>14</b> |
| 2.1      | Quel dialogue ? . . . . .                                       | 14        |
| 2.2      | From the simplest library to the "good" one . . . . .           | 14        |
| 2.3      | ESP is a HTTP GET client by <WiFi.h> . . . . .                  | 15        |
| 2.3.1    | Le code . . . . .                                               | 16        |
| 2.3.2    | L'analyse du code . . . . .                                     | 18        |
| 2.3.3    | TODO : Try it ! . . . . .                                       | 18        |
| 2.4      | ESP is a HTTP GET client by <HttpClient> . . . . .              | 20        |
| 2.4.1    | An example . . . . .                                            | 20        |
| 2.4.2    | HTTPS (optional) . . . . .                                      | 22        |
| 2.4.3    | TODO : Try it ! . . . . .                                       | 25        |
| 2.5      | ESP is a HTTP server . . . . .                                  | 26        |
| 2.5.1    | Version simpliste avec <WiFi.h> . . . . .                       | 26        |
| <b>3</b> | <b>ESP is an Asynchronous HTTP server</b>                       | <b>29</b> |
| 3.1      | Installation . . . . .                                          | 29        |
| 3.2      | Pour faire quoi? . . . . .                                      | 30        |
| 3.3      | La partie Html . . . . .                                        | 30        |
| 3.3.1    | Quelques commentaires ! . . . . .                               | 33        |
| 3.3.2    | PLACEHOLDERS . . . . .                                          | 33        |
| 3.3.3    | Grid HTML . . . . .                                             | 34        |
| 3.4      | Le Script JS . . . . .                                          | 34        |
| 3.5      | La partie "Periodic status report to" (début du TODO) . . . . . | 34        |
| 3.6      | Remarque "debugging" . . . . .                                  | 34        |
| 3.7      | La partie ESP . . . . .                                         | 35        |
| 3.7.1    | Le setup et la loop . . . . .                                   | 37        |
| 3.7.2    | Les routes et les Handlers . . . . .                            | 37        |
| 3.8      | Le delay, le yield et FreeRTOS . . . . .                        | 42        |
| <b>4</b> | <b>TODO : The final step of this lab !</b>                      | <b>44</b> |
| 4.1      | Cahier des charges de ce qui va être évalué ! . . . . .         | 44        |
| 4.1.1    | Configuration/administration de l'ESP . . . . .                 | 45        |
| 4.1.2    | Reporting de l'ESP . . . . .                                    | 45        |
| 4.1.3    | "Get a value" de l'ESP . . . . .                                | 46        |
| 4.1.4    | "Set a value" de l'ESP . . . . .                                | 46        |
| 4.2      | Dashboard . . . . .                                             | 47        |

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| 4.2.1    | Pour tester votre dashboard . . . . .   | 47        |
| <b>5</b> | <b>Annexes</b>                          | <b>48</b> |
| 5.1      | Quelques liens pour aider ... . . . .   | 48        |
| 5.2      | Exemple de formulaire en HTML . . . . . | 48        |

# 1 "Faire" du HTTP en ligne

Dans la suite du travail on va utiliser intensivement HTTP pour échanger des informations entre l'ESP et une machine "serveur".

Bien sûr, plus tard, on fera cela par la programmation, mais pour commencer nous allons **échanger en HTTP en utilisant la ligne de commande**.

- On peut ainsi se familiariser avec le protocole HTTP sans être perturbé par la programmation.
- Et même après, **cela permet de tester facilement ses requêtes avant d'en faire un code !**

Plusieurs outils existent pour "pratiquer" et mettre au point des API accessibles par HTTP.

- **soapdui** : très complet ... d'ailleurs trop complet pour nos besoins ?
- **postman** : idem !
- **cURL** : Acronyme de "client URL request library" (bibliothèque de requêtes aux URL pour les clients) ou encore "see URL" pour "voir l'URL".

## 1.1 cURL

cURL est une interface en ligne de commande **destinée à récupérer le contenu d'une ressource accessible par un réseau informatique**.

- cURL supporte notamment les protocoles DICT, file, FTP, FTPS, Gopher, HTTP, HTTPS, IMAP, IMAPS, LDAP, LDAPS, POP3, POP3S, RTSP, SCP, SFTP, SMB, SMBS, SMTP, SMTPS, Telnet et TFTP.
- MAIS SURTOUT, cURL permet de créer ou modifier une ressource (contrairement à wget), **il peut ainsi être utilisé en tant que client REST**.

Références :

- <https://everything.curl.dev/http>
- <https://www.it-connect.fr/curl-loutil-testeur-des-protocoles-divers/>
- Des exemples ... <https://linuxize.com/post/curl-rest-api/>
- ...

## 1.2 First request

La ressource que l'on souhaite accéder est désignée à l'aide d'une URL et doit être d'un type supporté.

```
curl [options] <url>
```

---

Par exemple pour voir le contenu d'une URL "simple" de la forme **http://host:port/path**, on procédera ainsi :

```
curl http://httpbin.org/ip
```

répondra

```
{  
  "origin": "134.59.131.45"  
}
```

ou

```
curl http://httpbin.org/get
```

répondra

```
{  
  "args": {},  
  "headers": {  
    "Accept": "*/*",  
    "Host": "httpbin.org",  
    "User-Agent": "curl/7.68.0",  
    "X-Amzn-Trace-Id": "Root=1-6310e439-36f816f037654abc7c97b689"  
  },  
  "origin": "109.210.116.27",  
  "url": "http://httpbin.org/get"  
}
```

### 1.3 "httpbin.org" : Test Server

Le site <http://httpbin.org/> est un site de "test" bien connu.

- La plupart du temps, il "**fait écho**" aux données utilisées dans votre requête.

Selon ce qui vous intéresse, vous choisissez la bonne URL :

**http://httpbin.org/anything** : Renvoie la plupart des éléments ci-dessous.

**http://httpbin.org/ip** : Renvoie l'adresse IP d'origine.

**http://httpbin.org/user-agent** : Renvoie l'agent utilisateur.

**http://httpbin.org/headers** : Renvoie le dictionnaire représentant l'en-tête.

**http://httpbin.org/get** : Renvoie les données de la requête GET.

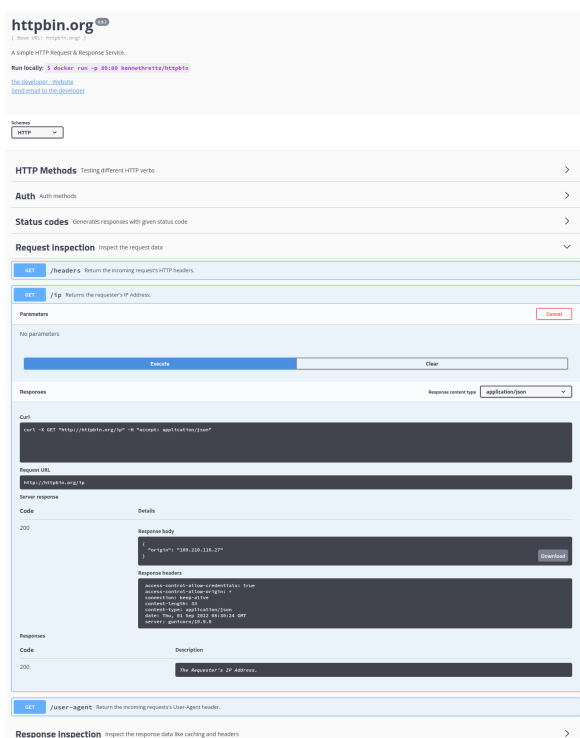
**http://httpbin.org/post** : Renvoie les données de la requête POST.

• • •

• • •

Il y a un site Web associé à chaque URL qui vous permet d'utiliser une UI pour produire une requête et analyser sa réponse :

- [http://httpbin.org/#/Request\\_inspection/get\\_ip](http://httpbin.org/#/Request_inspection/get_ip)



Si ce site ne vous plaît pas ... vous pouvez en trouver d'autres sur :

<https://mixedanalytics.com/blog/list-actually-free-open-no-auth-needed-apis/>

## 1.4 See more of "request / response" exchange

Avec l'option "verbose" (-v), cURL, montre les headers des messages (request/response) échangés :

```
curl -v http://httpbin.org/ip
* Trying 3.209.149.47:80...
* TCP_NODELAY set
* Connected to httpbin.org (3.209.149.47) port 80 (#0)
> GET /ip HTTP/1.1
> Host: httpbin.org
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Wed, 06 Oct 2021 12:55:46 GMT
< Content-Type: application/json
< Content-Length: 32
< Connection: keep-alive
< Server: gunicorn/19.9.0
< Access-Control-Allow-Origin: *
< Access-Control-Allow-Credentials: true
<
{
  "origin": "134.59.131.45"
}
* Connection #0 to host httpbin.org left intact
```

Vous remarquez les caractères ">" pour la requête et "<" pour la réponse.



## 1.5 Few IMPORTANT examples

On donne 4 utilisations MAJEURES de cURL ... il faut les comprendre !

### 1.5.1 "GET request" WITH parameters

"Par défaut" cURL formule une requête GET ... c'est ce que l'on voit sur l'exemple précédent.

Dans ce qui suit, on complexifie les requêtes en utilisant des URLs **avec un ou des paramètres** :

**http://host:port/path?query\_string**

La "query string" qui suit le point d'interrogation est **un ensemble de paires** :

nom du paramètre=valeur du paramètre.

Ces paramètres seront utilisés pour donner des informations à la machine qui reçoit et analyse la requête HTTP.

- C'est le principe des "API", du REST, ...

Quelques exemples de requêtes cURL GET **avec paramètres** :

*#Deux paramètres : led1 et led2*

```
curl -v http://httpbin.org/get?led1="OFF"&led2="ON"
```

*#Exemple qui ne marche plus ... Google's Terms of Service :-(*

*#Google propose désormais "son API" pour faire des recherches à*

*#partir de scripts ! Payant ?*

```
curl -v "https://www.google.com/search?q=esp32"
```

*#Exemple qui ne marche pas ... mais Google vous le dit et renvoie*

*#les Google's Terms of Service :-(*

```
curl -v "https://www.google.com/search?channel=fs&client=ubuntu&q=esp32"
```

*#Vous pouvez néanmoins essayer cette requête (sans le curl) dans un navigateur*

*# => et voir que le "user agent" joue son rôle.*

### 1.5.2 "GET request" WITH parameters AND headers

Dans la requête qui suit, on ajoute un header à la requête cURL :

1. on spécifie (inutilement et d'ailleurs il nous le dit) l'utilisation du verbe GET,
2. et on indique dans le header de cette requête que l'on souhaite une réponse en JSON.

---

```
curl -v -X GET http://httpbin.org/get?led1="OFF"&led2="ON" \
-H "accept: application/json"
Note: Unnecessary use of -X or --request, GET is already inferred.
* Trying 54.159.86.231:80...
* TCP_NODELAY set
* Connected to httpbin.org (54.159.86.231) port 80 (#0)
> GET /get?led1=OFF&led2=ON HTTP/1.1
> Host: httpbin.org
> User-Agent: curl/7.68.0
> accept: application/json
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Tue, 26 Oct 2021 13:42:51 GMT
< Content-Type: application/json
< Content-Length: 324
< Connection: keep-alive
< Server: gunicorn/19.9.0
< Access-Control-Allow-Origin: *
< Access-Control-Allow-Credentials: true
<
{
  "args": {
    "led1": "OFF",
    "led2": "ON"
  },
  "headers": {
    "Accept": "application/json",
    "Host": "httpbin.org",
    "User-Agent": "curl/7.68.0",
    "X-Amzn-Trace-Id": "Root=1-617805db-7a514add77eae01549e5758"
  },
  "origin": "134.59.131.45",
  "url": "http://httpbin.org/get?led1=OFF&led2=ON"
}
* Connection #0 to host httpbin.org left intact
```

---

La réponse du site httpbin.org montre bien qu'il a "su" identifier les paramètres led1 et led2 (avec leurs valeurs) de la requête,  
...et il répond en Json.

### 1.5.3 "POST request" WITHOUT parameter

On change le verbe de la requête et comme on souhaite voir le retour de la requête POST on change aussi l'URL du site "httpbin.org".

"http://httpbin.org/post" permet d'analyser les requêtes POST :

---

```
curl -v -X POST http://httpbin.org/post
* Trying 3.209.149.47:80...
* TCP_NODELAY set
* Connected to httpbin.org (3.209.149.47) port 80 (#0)
> POST /post HTTP/1.1
> Host: httpbin.org
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Wed, 06 Oct 2021 13:03:10 GMT
< Content-Type: application/json
< Content-Length: 318
< Connection: keep-alive
< Server: gunicorn/19.9.0
< Access-Control-Allow-Origin: *
< Access-Control-Allow-Credentials: true
<
{
  "args": {},
  "data": "",
  "files": {},
  "form": {},
  "headers": {
    "Accept": "*/*",
    "Host": "httpbin.org",
    "User-Agent": "curl/7.68.0",
    "X-Amzn-Trace-Id": "Root=1-615d9e8e-4a26394d221bb040307ec83e"
  },
  "json": null,
  "origin": "134.59.131.45",
  "url": "http://httpbin.org/post"
}
* Connection #0 to host httpbin.org left intact
```

---

Pour l'instant la requête n'a pas de donnée, donc le "data" est la chaîne vide.

- Normal !

#### 1.5.4 "POST request" WITH parameters AND JSON body

Un des différences entre GET et POST est que l'on peut associer un corps avec des données à une requête POST.

- Ce corps permettra de formuler un message dans une syntaxe plus libre que celle des paramètres de la requête.

Le format du corps est spécifié par l'attribut "Content-Type" de l'entête.

- Nous, on est plutôt "précis" en disant que c'est du Json (puisque cela en est !).

Souvent, les applications utilisent des types de media

- "application/x-www-form-urlencoded"
- ou "multipart/form-data"
- ou "text/plain"
- ou ...

<https://stackoverflow.com/questions/9870523/what-are-the-differences-between-application-json-and-application-x-www-form-url>

Dans tous les cas, l'application "server" doit effectuer les étapes suivantes :

1. Lire le champ Content-Type
2. Si la valeur ne correspond pas à l'un des types de média pris en charge, renvoyer une réponse avec un code d'état 415.
3. sinon, décoder les valeurs du corps du message.

La commande cURL suivante permet **de formuler une requête POST**

- avec des paramètres
- ET un corps en JSON

---

```
curl -i -X POST -H "Content-Type: application/json" \  
-d '{"temp": "23", "light": "1234"}' \  
http://httpbin.org/post?led1="OFF"&led2="ON"
```

```
HTTP/1.1 200 OK  
Date: Thu, 01 Sep 2022 09:09:40 GMT  
Content-Type: application/json  
Content-Length: 467  
Connection: keep-alive  
Server: gunicorn/19.9.0  
Access-Control-Allow-Origin: *
```

Access-Control-Allow-Credentials: **true**

```
{
  "args": {
    "led1": "OFF",
    "led2": "ON"
  },
  "data": "{temp:23, light:1234}",
  "files": {},
  "form": {},
  "headers": {
    "Accept": "*/*",
    "Content-Length": "21",
    "Content-Type": "application/json",
    "Host": "httpbin.org",
    "User-Agent": "curl/7.68.0",
    "X-Amzn-Trace-Id": "Root=1-631076d4-4862d1021264fb87153d6716"
  },
  "json": null,
  "origin": "109.210.116.27",
  "url": "http://httpbin.org/post?led1=OFF&led2=ON"
}
```

---

Ok !

- le champ data est conforme à ce que la requête a envoyé.
- Les args correspondent aussi.

## 1.6 TODO : cURL

- Just try ...

## 2 HTTP as application protocol

Maintenant que la liaison au réseau est réalisée, on va pouvoir dialoguer avec l'ESP.

- Pour l'instant, on souhaite baser ces dialogues sur le protocole HTTP.

### 2.1 Quel dialogue ?

Cette notion de "dialogue" est assez large et peut utiliser des architectures bien différentes :

- Le "serveur central" est dans une démarche où il récupère les informations du monde réel par exemple par "polling" ... et l'ESP ne ferait alors que répondre à des requêtes.
- Ou alors, l'ESP choisit ses instants de communications. C'est lui qui démarre le dialogue et le serveur (au centre du réseau) est en attente.
- Ou alors, ...

---

### 2.2 From the simplest library to the "good" one

On va utiliser des bibliothèques qui permettent de disposer de la faculté d'émettre et de recevoir des requêtes/réponses HTTP.

Le "problème" c'est qu'il y en a pas mal qui font "presque" la même chose ... avec des interfaces "**presques**" identiques ... mais pas tout à fait :-)

- C'est un peu pénible !

J'en ai choisi quelques unes ... et je tente d'illustrer progressivement la construction des requêtes/réponses HTTP.

## 2.3 ESP is a HTTP GET client by <WiFi.h>

On vient de voir dans la partie WiFi que cette bibliothèque <WiFi.h> est majeure dans l'établissement de la connexion au réseau.

Mais cette bibliothèque **offre plus** que l'établissement d'un lien WiFi. Elle "déborde" largement et propose des fonctionnalités de type sockets :

- la spécification d'un port (couche transport),
- l'établissement d'une connexion,
- ...

Dans ce premier exemple, **on veut réaliser depuis l'ESP une requête GET sur un serveur HTTP** :

- L'ESP **est donc le client**.
- Pour commencer, le serveur Web ciblé est la machine : `httpbin.org`
- La requête est "toute simple" puisqu'elle ne contient pas de paramètres, juste l'URL : `http://httpbin.org/ip`

La page HTML, produite en réponse par ce serveur, contient l'adresse IP de la machine qui a formulé la requête ... (donc de l'ESP).

Pour savoir ce que l'ESP va recevoir en réponse, il suffit de se rappeler du résultat de `cURL` :

```
~> curl http://httpbin.org/ip
{
  "origin": "134.59.131.45"
}
```

et éventuellement utiliser l'option de verbose si vous voulez en savoir plus sur la requête effectuée.

Pour réaliser cette requête GET, **on utilise dans un premier temps l'objet WiFiClient de <WiFi.h>**

Cet objet `WiFiClient` s'apparente donc à une "socket HTTP" :

1. Il permet de se connecter et d'échanger en HTTP.
2. Par contre, **il n'aide en rien sur la construction syntaxique du message** et donc la requête doit être composée explicitement comme une chaîne de caractères.

### 2.3.1 Le code

```
1  /*
2   * Fichier : http_by_wificlient/http_by_wificlient.ino
3   * Auteur : G.Menez based on Rui Santos https://randommerdtutorials.com/esp32-http-get-post-arduino/
4   * Send a HTTP GET request without params from ESP.
5   */
6  #include <WiFi.h>
7  #include "wifi_utils.h"
8  #define USE_SERIAL Serial
9  /*=====*/
10 void httpGETRequest(const char* httpServer, const int httpPort = 80) {
11  /*
12   * This method sends data via HTTP GET requests to host/url
13   * and prints on the console the response/payload of the GET request to httpServer.
14   */
15  WiFiClient client; //https://www.arduino.cc/reference/en/libraries/wifi/
16  USE_SERIAL.printf("=====\\n");
17
18  /* 1) Connection -----*/
19  // Use WiFiClient class to create TCP connections
20  if (client.connect(httpServer, httpPort) == 0) {
21    USE_SERIAL.printf("Requesting URL : %s => failed !\\n", httpServer);
22    return;
23  }
24  else{
25    USE_SERIAL.printf("URL %s connected !\\n", httpServer);
26  }
27  // BOF BOF ... meme avec un reseau out ...
28  // la demande de connexion avec l'httpServer rend 1 => Bizarre !!
29
30  /* 2) Payload building -----*/
31  // Now create a URI for the HTTP request : can contain the
32  // information we want to send to the server => GET style !!
33  // String url = "/api/timezone/Europe/Paris";
34  String url = "/ip";
35  String req = String("GET "); // Now create HTTP request header INCLUDING URL
36  req += url + " HTTP/1.1\\r\\n";
37  req += "host: " + String(httpServer) + "\\r\\n";
38  req += "User-Agent: esp-idf/1.0 esp32 \\r\\n";
39  req += "Accept: */* \\r\\n";
40  req += "Connection: close \\r\\n";
41  req += "\\r\\n"; // empty line : separator header/body
42
43  USE_SERIAL.printf("-----\\n");
44  USE_SERIAL.printf("Request : \\n %s", req.c_str());
45  USE_SERIAL.printf("-----\\n");
46
47  //USE_SERIAL.printf("Send request to URL : %s\\n", url.c_str());
48  /* 3) Send request to httpServer through client socket -----*/
49  client.print(req); // Send !
50  /* 4) Wait WITH timeout for server response -----*/
51  // https://www.arduino.cc/en/Reference/WiFiClientAvailable
52  unsigned long timeout = millis();
53  while (client.available() == 0) { // no answer => timeout mechanism !
54    if (millis() - timeout > 10000) {
55      USE_SERIAL.println(">>> Client Timeout ! => Disconnection !");
```



```

56     client.stop(); // disconnect
57     return;
58 }
59 }
60 // 5) Read and print response
61 USE_SERIAL.println("Response : ");
62 // all the lines of the reply from server and print them to Serial
63 while (client.available()) { // Returns the number of bytes available for reading
64     String line = client.readStringUntil('\r');
65     USE_SERIAL.print(line); // echo to console "this" line
66
67     // en version car by car
68     //char c = client.read();
69     //USE_SERIAL.print(c);
70 }
71
72 // End connection and Free resources
73 USE_SERIAL.println("Closing connection from client side");
74 client.stop(); // disconnect
75 }
76 /*=====*/
77 /*---- Arduino IDE paradigm : setup+loop ----*/
78 /*=====*/
79 void setup() {
80     /* Serial connection -----*/
81     USE_SERIAL.begin(9600);
82     while(!USE_SERIAL); //wait for a serial connection
83     /* WiFi connection -----*/
84     String hostname = "Mon petit objet ESP32";
85     wifi_connect_multi(hostname);
86     /* WiFi status -----*/
87     if (WiFi.status() == WL_CONNECTED){
88         USE_SERIAL.print("\nWiFi connected : yes ! \n");
89         wifi_printstatus(0);
90     }
91     else {
92         USE_SERIAL.print("\nWiFi connected : no ! \n");
93         // ESP.restart();
94     }
95     /* Install OTA -----*/
96     //setupOTA("GM_ESP");
97
98     // your code
99 }
100 void loop(){
101     // IPAddress httpServerip(216,58,205,195); // Google.com IP
102     const char httpServer[100] = "httpbin.org"; // "http://worldtimeapi.org";
103     const int httpPort = 80;
104     USE_SERIAL.println("Sending HTTP Request ...");
105     httpGETRequest(httpServer, httpPort);
106     USE_SERIAL.println("Done !");
107     delay(10000); //ms
108 }

```

### 2.3.2 L'analyse du code

La **construction du message/payload** (vers ligne 34) contenant la requête est conforme au protocole HTTP :

1. **A message has a header part and a message body separated by a blank/empty line.**  
The blank line is ALWAYS needed **even if there is no** message body.
2. The header starts with a command (here "GET") and has additional lines of key value pairs separated by a colon and a space.
3. The "Connection: Close" header tells the server to close the connection after the request has been processed.

You could have alternatively send Connection: Keep-Alive if you wanted the connection to be kept alive after the request was processed.

**Dans cet exemple**, on se connecte à CHAQUE requête ! ... pas très performant ?

4. Accept: \*/\*  
Client will accept any MIME type.
5. Lines in the header and the blank line at the end of the header **must end with a carriage return and linefeed pair**

CRLF = `\r\n`.

<https://stackoverflow.com/questions/5757290/http-header-line-break-style>

6. If there is a message body, it can be anything you want it to be.

**Ligne 49, on envoie cette requête (par un print), puis on attend une réponse et si elle arrive on la lit.**

### 2.3.3 TODO : Try it !

Pour l'instant la requête est simple et la réponse est affichée sur la console de l'ESP :

```
10:48:08.848 -> connecting to httpbin.org
10:48:09.080 -> Requesting URL: /ip
10:48:09.512 -> HTTP/1.1 200 OK
10:48:09.512 -> Date: Wed, 19 Aug 2020 08:48:09 GMT
10:48:09.545 -> Content-Type: application/json
10:48:09.578 -> Content-Length: 32
10:48:09.611 -> Connection: close
10:48:09.644 -> Server: gunicorn/19.9.0
10:48:09.644 -> Access-Control-Allow-Origin: *
10:48:09.677 -> Access-Control-Allow-Credentials: true
10:48:10.606 ->
10:48:10.606 -> {
10:48:10.606 ->   "origin": "80.214.145.35"
10:48:10.639 -> }
10:48:10.639 ->
10:48:10.639 -> closing connection
```

Il est intéressant de constater la similitude que l'on a implémenté entre "notre" requête et celle affichée par cURL.

- En ce sens, il peut être souvent utile de comparer les requêtes que l'on fabrique avec celles de cURL.

## 2.4 ESP is a HTTP GET client by <HTTPClient>

Pour exprimer que l'on souhaite faire une requête "GET", **sans pour autant devoir l'écrire complètement** (comme on vient de le faire), on peut utiliser la bibliothèque `HTTPClient`

- le HTTP en majuscules ... sinon ce n'est pas la même :-)

<https://github.com/espressif/arduino-esp32/tree/master/libraries/HTTPClient/examples>

Cette bibliothèque développe des objets "plus" spécialisés pour HTTP et facilite l'utilisation des requêtes.

- Elle permet aussi d'associer (facilement) le certificat du serveur à la requête et donc de travailler en https !

### 2.4.1 An example

Dans l'exemple qui suit, on utilise cette bibliothèque pour **envoyer périodiquement des requêtes GET à un serveur**.

- Ces requêtes sont conformes à ce que l'on pourrait envoyer avec la commande cURL suivante :

```
curl -v http://httpbin.org/get?led1="OFF"&led2="ON"
```

Ce qui pourrait permettre à l'ESP d'indiquer le status de ses LEDs à un serveur ...

```
1  /*
2  * Fichier : http_by_HTTPClient/http_by_HTTPClient.ino
3  * Auteur : G.Menez
4  => based on
5  https://github.com/espressif/arduino-esp32/blob/master/libraries/HTTPClient/examples/BasicHttpClient/BasicHttpClient.ino
6  */
7  #include <WiFi.h>
8  #include "wifi_utils.h"
9  #include <HTTPClient.h>
10 // the following variables are unsigned longs because the time, measured in
11 // milliseconds, will quickly become a bigger number than can be stored in an int.
12 unsigned long lastTime = 0;
13 // Set timer
14 unsigned long loop_period = 10L * 1000; /* => 10000ms : 10 s */
15
16 #define USE_SERIAL Serial
17 /*=====*/
18 String httpGETRequest(const char* UrlServer) {
19     /*
20      * This method sends data via HTTP GET requests to host/url
21      * and return the response/payload of the GET request to UrlServer
22      */
23     String payload = "{}";
24     HTTPClient http; // http protocol entity => client
25
26     USE_SERIAL.printf("Requesting URL : %s\n", UrlServer);
```

```

27
28 // Configure traged server and url
29 http.begin(UrlServer);
30
31 // Connect and Send HTTP request
32 int httpResponseCode = http.GET();
33
34 // Get the response => will fill payload String
35 if (httpResponseCode>0) {
36     USE_SERIAL.printf("HTTP Response code : %d", httpResponseCode);
37     payload = http.getString();
38 }
39 else {
40     USE_SERIAL.printf("Error code on HTTP GET Request : %d", httpResponseCode);
41 }
42 // End connection and Free resources
43 http.end();
44
45 return payload;
46 }
47 /*=====*/
48 /*---- Arduino IDE paradigm : setup+loop ----*/
49 /*=====*/
50 void setup() {
51     /* Serial connection -----*/
52     USE_SERIAL.begin(9600);
53     while(!USE_SERIAL); //wait for a serial connection
54
55     /* WiFi connection -----*/
56     String hostname = "Mon petit objet ESP32";
57     wifi_connect_multi(hostname);
58
59     /* WiFi status -----*/
60     if (WiFi.status() == WL_CONNECTED){
61         USE_SERIAL.print("\nWiFi connected : yes ! \n");
62         wifi_printstatus(0);
63     }
64     else {
65         USE_SERIAL.print("\nWiFi connected : no ! \n");
66         // ESP.restart();
67     }
68
69     /* Install OTA -----*/
70     //setupOTA("GM_ESP");
71
72     // your code
73 }
74
75 void loop() {
76     char host[100] = "http://httpbin.org"; // = "http://worldtimeapi.org/";
77     const int httpPort = 80;
78
79     // Make Url
80     String path = "/get";
81     String params = "?led1=\"OFF\"&led2=\"ON\"";
82     String url = String(host)+path+params;

```

```

83
84 //Send an HTTP request every loop_period in ms
85 if ((millis() - lastTime) > loop_period) {
86     //Check WiFi connection status
87     if(WiFi.status()== WL_CONNECTED){
88
89         String ret = httpGETRequest(url.c_str());
90         USE_SERIAL.println(ret);
91     }
92     else {
93         USE_SERIAL.println("WiFi Disconnected");
94     }
95     lastTime = millis();
96 }
97 }

```

Cet exemple est **important** car il est représentatif d'un comportement possible de l'objet dans le réseau.

Ici, le dialogue est à l'initiative de l'ESP :

1. Envoi périodique d'information/ des valeurs des capteurs à un serveur (situé plus au centre du réseau).
2. Utilisation d'une requête GET du protocole HTTP via la fonction `httpGETRequest(...)`
3. Dans ce code, bien que l'on ait accès à la charge/payload de la réponse à cette requête, nous ne l'utilisons pas.

On pourrait sans doute en faire quelque chose !?

On comprend aussi que la transmission de données par ce type de requête GET peut devenir lourde et ne convient pas si on veut formater le message en JSON et le mettre dans la "payload" de la requête.

- Il va falloir faire des requêtes POST !

#### 2.4.2 HTTPS (optional)

La sécurité des échanges dans le contexte de l'IoT (comme dans d'autres contextes) est fondamentale ... mais on manque de temps :-)

L'exemple qui suit montre comment passer en https.

Après avoir récupéré le certificat du serveur avec lequel on veut travailler : `www.howsmyssl.com`

```
openssl s_client -showcerts -connect www.howsmyssl.com:443
```

- The correct cert in this example it to pick the 3rd one in the chain, with "i:0 = Digital Signature Trust Co." as it is the parent certificate !

On utilise ces sketches pour réaliser une requête GET avec TLS.

Avec curl, ça fonctionne aussi (si on a placé le certificat dans le fichier ".pem") :

```
curl --cacert www-howsmyssl-com.pem https://www.howsmyssl.com/a/check
```

```
1  /*
2   * Fichier : https_by_HTTPClient/https_by_HTTPClient.ino
3   * Auteur : G.Menez
4
5   => based on Rui Santos work https://randommerdtutorials.com/esp32-http-get-post-arduino/
6       based on https://github.com/espressif/arduino-esp32/blob/master/libraries/HttpClient/examples/BasicHttpClient/BasicHttp
7   */
8   #include <WiFi.h>
9   #include "wifi_utils.h"
10  #include <HttpClient.h>
11
12  // the following variables are unsigned longs because the time, measured in
13  // milliseconds, will quickly become a bigger number than can be stored in an int.
14  unsigned long lastTime = 0;
15  // Set timer
16  unsigned long loop_period = 10L * 1000; /* => 10000ms : 10 s */
17
18  #define USE_SERIAL Serial
19  const static char* ca PROGMEM = \
20  "-----BEGIN CERTIFICATE-----\n" \
21  "MIIFYDCCBeigAwIBAgIQAF3ITfU6UK47naqPGQKtzANBgkqhkiG9w0BAQsFADA/\n" \
22  "MSQwIgYDVQQKEExtEaWdpdGFsIFNpZ25hdHVyZSBUcnVzdCBDb250YXZAVBGNBAMT\n" \
23  "DkRtVVCBSb290IENBIFgzMB4XDTEyMDEyMTQwM1oXDTEyMDkzMDE4MTQwM1ow\n" \
24  "TzELMAkGA1UEBhMCVGVybmV0IFN1Y3VyaXR5IFJlc2Vh\n" \
25  "cmNoIEEdyb3VwMRUwEwYDVQDEwxxJU1JHIFJvb3QwWDEwggIiMAOGCSqGSIb3DQEB\n" \
26  "AQUAA4ICDwAwggIKAoICAQCt6CRz9BQ385ueK1coHie+3Lff0JCMbjzmV6B493XC\n" \
27  "ov71am72AE8o295ohmxEk7axY/OUEmu/H9LqMZshftEzPLpI9d153704/xLxIZpL\n" \
28  "wYqGcWLKZmZs348cL+tKSIG8+TA5oCu4kuPt51+1A0f00eXfJl111Po0K5PCm+D\n" \
29  "LtfJV4yAdLbaL9A4jXsDcCEbdfIwPPqPrt3aY6vrfK/CjhFLfs8L6P+1dy70sntK\n" \
30  "4EwSJQwJMpOFTJ0wT2e4ZvxCzSow/iaNhUd6shweU9GNx7C7ib1uYgeGJXDR5\n" \
31  "bHbv05BieebbpJovJsXQE0E03tkQjhb7t/eo98f1AgeYjzYllefIn5YNNnWe+w5y\n" \
32  "sR2bvAP5SQYgd0FtCrWQemsAXaVCg/Y39W9Eh81LygXbNKYwagJZHduRze6zqxZ\n" \
33  "Xmidf3LWicUGQSk+WT7dJvUkyRGnWqNMQB9GoZm1pZpRboY7nn1ypxIFeFntP1F4\n" \
34  "FQsDj43QLwYpntKHETzBRL8xurgUBN8Q5N0s8p0544fAQjQMNrbCta0B7rBMDbC\n" \
35  "SLeC05imfWCKoMpgsy6vYMEG6KDA0Gh1gXxG8K28Kh8hjtGqEgqiNx2mna/H2q1\n" \
36  "PRmP6zjzZN7IKw0KKP/32+IVQtQi0Cdd4Xn+G0dwiK105tmL0sbdJ1Fu/7xk9TND\n" \
37  "TwIDAQABo4IBRjCCAUIwDwYDVROTAQH/BAUwAwEB/zAOBgNVHQ8BAf8EBAMCAQYw\n" \
38  "SwYIKwYBBQUHAQEPPzA9MDsGCCsGAQUFBzACChi9odHRwOi8vYXBwcy5pZGVudHJ1\n" \
39  "c3QyY29tL3Jvb3RzL2RzdHJvb3RjYXgzLnA3YzAfBgNVHSMEGDAWgBTEp7Gkeyxx\n" \
40  "+tvhS5B1/8QVYIwJEDBUBgNVHSAETTBLMAgGBmeBDAECATA/BgsrBgEEAYLfEwEB\n" \
41  "ATAwMC4GCCsGAQUFBwIBFiJodHRwOi8vY3BzLnJvb3QteDEubGV0c2VuY3J5c3R5cGU\n" \
42  "b3JnMDwGA1UdHwQ1MDMwMmAvOjC2GK2h0dHA6Ly9jcmwuaW1bnRydXN0LmNvbS9E\n" \
43  "U1RST09UQ0FYM0NSTC5jcmwwHQYDVRO0BBYEFHm0WeZ7tuXkAXOACIjIGlj26Ztu\n" \
44  "MAOGCSqGSIb3DQEBChUAA4IBAQAkwBslm7/D1LQrt2M51oGrS+o44+/yQoDFVDC\n" \
45  "5WxCu2+b9LRPwkSICHXM6webFGJueN7sJ7o5XPWioW5W1HAQU7G75K/QosMrAdSW\n" \
46  "9MUgNTP52GE24HGNTLi1qoJF1cDyqSMo59ahy2cI2qBDLKobkx/J3vWraVOT9VuG\n" \
47  "WCLKTVXkcGdtwlfFRjlBz4pyYg1htmf5X6DY08A4jqv2I19DjXA6USbW1FzXSLr90\n" \
48  "he8Y4IWS6wY7bCkjcWDCRQJMEhg76fs03txE+FiYruq9RUWhiF1myv4Q6W+CyBFC\n" \
49  "Dfvp700GAN6EOM4+qR9sdjoSYKEBpsr6GtPAQw4dy753ec5\n" \
50  "-----END CERTIFICATE-----\n";
51
52  /*=====*/
53  String httpGetRequest(const char* UrlServer, const char* ca) {
54  /*
```

```

55  * This method sends data via HTTP GET requests to host/url
56  * and return the response/payload of the GET request to UriServer
57  */
58
59  String payload = "{}";
60  HTTPClient http; // http protocol entity => client
61
62  USE_SERIAL.printf("Requesting URL : %s\n", UriServer);
63
64  // Configure traged server and url
65  http.begin(UriServer, ca);
66
67  // Connect and Send HTTP request
68  int httpResponseCode = http.GET();
69
70  // Get the response => will fill payload String
71  if (httpResponseCode>0) {
72      USE_SERIAL.printf("HTTP Response code : %d", httpResponseCode);
73      payload = http.getString();
74  }
75  else {
76      USE_SERIAL.printf("Error code on HTTP GET Request : %d", httpResponseCode);
77  }
78  // End connection and Free resources
79  http.end();
80
81  return payload;
82 }
83 /*=====*/
84 /*---- Arduino IDE paradigm : setup+loop ----*/
85 /*=====*/
86 void setup() {
87     /* Serial connection -----*/
88     USE_SERIAL.begin(9600);
89     while(!USE_SERIAL); //wait for a serial connection
90
91     /* WiFi connection -----*/
92     String hostname = "Mon petit objet ESP32";
93     wifi_connect_multi(hostname);
94
95     /* WiFi status -----*/
96     if (WiFi.status() == WL_CONNECTED){
97         USE_SERIAL.print("\nWiFi connected : yes ! \n");
98         wifi_printstatus(0);
99     }
100    else {
101        USE_SERIAL.print("\nWiFi connected : no ! \n");
102        // ESP.restart();
103    }
104    /* Install OTA -----*/
105    //setupOTA("GM_ESP");
106
107    // your code
108 }
109
110 void loop() { // cf https://www.howsmyssl.com/s/api.html

```



```

111     char host[100] = "https://www.howsmyssl.com";
112     const int httpPort = 443;
113     // Make Url
114     String path = "/a/check";
115     String params = "";
116     String url = String(host)+path+params;
117     //Send an HTTP request every loop_period in ms
118     if ((millis() - lastTime) > loop_period) {
119         //Check WiFi connection status
120         if(WiFi.status()== WL_CONNECTED){
121
122             String ret = httpGETRequest(url.c_str(),ca);
123             USE_SERIAL.println(ret);
124         }
125         else {
126             USE_SERIAL.println("WiFi Disconnected");
127         }
128         lastTime = millis();
129     }
130 }

```

### 2.4.3 TODO : Try it !

- Try these examples !

On vient de montrer comment on peut utiliser les ESP **en tant que clients** HTTP ...capables d'émettre des requêtes vers un "serveur".

## 2.5 ESP is a HTTP server

**On va maintenant en faire des serveurs HTTP** ...capables de **répondre** à des requêtes venant de clients !

### 2.5.1 Version simpliste avec <WiFi.h>

On commence par réaliser un serveur HTTP "simple".

- Il est simple car il ne gère/analyse pas les "paths" des URLs.
- En fait, dès qu'il (dans la fonction `loop`) reçoit une demande de connexion sur le port, il affiche la requête sur la console puis émet une réponse qui contient un "morceau" d'HTML qui contiendra le "uptime" en millisecondes.

Donc, si un navigateur (qui serait le client) se connecte sur ce serveur ( l'URL correspond à son IP), on devrait voir afficher du HTML !

#### Attention !!

- Tout le monde, client et serveur, doit être joignable et donc soyez attentif aux réseaux que vous utilisez !
- Ce serveur est construit sur la bibliothèque <WiFi.h> => qui offre les objets `WiFiClient` et `WiFiServer`.

C'est vraiment simpliste, puisque qu'on n'analyse PAS au niveau de l'ESP l'URL formulée par la requête.

- On répond à une connexion d'un client (sans doute un navigateur) sans lui permettre de choisir une action.

De plus la réponse HTTP est construite "à la main" dans la fonction `httpReply`.

- Elle contient la fabrication du header de la réponse ET le message : c'est lourd !

---

```
1  /*
2   * Fichier : http_as_server/http_as_server.ino
3   * Auteur : G.Menez
4   */
5  #include <WiFi.h>
6  #include "wifi_utils.h"
7
8  // the following variables are unsigned longs because the time, measured in
9  // milliseconds, will quickly become a bigger number than can be stored in an int.
10 unsigned long lastTime = 0;
11 // Set timer
12 unsigned long loop_period = 1L * 1000; /* => 1000ms : 1 s */
```

```

13
14 #define USE_SERIAL Serial
15
16 // Instanciation of a "Web server" on port 80
17 WiFiServer server(80);
18
19 /*=====*/
20
21 void httpReply(WiFiClient client) {
22     // this method makes a simple HTTP response to a GET request
23     // the body syntax is HTML => supposed to be displayed by a navigator
24     client.println("HTTP/1.1 200 OK");
25     client.println("Content-Type: text/html");
26     client.println("Connection: close"); // the connection will be closed after
27                                         // completion of the response
28     client.println("Refresh: 5");        // refresh the page automatically every 5 sec
29
30     client.println(); // EMPTY LINE between header and body
31
32     client.println("<!DOCTYPE HTML>");
33     client.println("<html>");
34     client.print("Hello, je tourne depuis : "); // Returns the ms passed since the ESP
35                                                  // began running the current program.
36     client.print(millis()/1000); // On pourrait sans doute donner une info
37     client.println("s <br />"); // plus pertinente ? temperature ?
38     client.println("</html>");
39 }
40
41 /*=====*/
42 /*---- Arduino IDE paradigm : setup+loop ----*/
43 /*=====*/
44 void setup(){
45     /* Serial connection -----*/
46     USE_SERIAL.begin(9600);
47     while(!USE_SERIAL); //wait for a serial connection
48     /* WiFi connection -----*/
49     String hostname = "Mon petit objet ESP32";
50     wifi_connect_multi(hostname);
51     /* WiFi status -----*/
52     if (WiFi.status() == WL_CONNECTED){
53         USE_SERIAL.print("\nWiFi connected : yes ! \n");
54         wifi_printstatus(0);
55     }
56     else {
57         USE_SERIAL.print("\nWiFi connected : no ! \n");
58         // ESP.restart();
59     }
60     /* Lancement du serveur HTTP */
61     server.begin();
62 }
63
64 void loop() { /* Requests processing */
65     WiFiClient client = server.available(); // listen for incoming clients
66     if (client) { // incoming client
67         USE_SERIAL.println("New client is connecting !");
68

```

```

69     boolean currentLineIsBlank = true; // an http request ends with a blank line CRLF
70     while (client.connected()) {
71         if (client.available()) {
72             char c = client.read();
73             USE_SERIAL.write(c); // ECHO the request (char by char) on the console
74
75             if (c == '\n' && currentLineIsBlank) { // if you've gotten to a CRLF the http GET request has ended,
76                 httpReply(client); // so you can send the REPLY
77                 break;
78             }
79             if (c == '\r') { // you're starting a new line
80                 currentLineIsBlank = true;
81             } else if (c != '\r') { // you've gotten a character on the current line
82                 currentLineIsBlank = false; // this is not a blank line
83             }
84         } // end if
85     } // end while
86
87     // give the web browser time to receive the data
88     delay(loop_period); // ms
89
90     // close the connection :
91     client.stop();
92     USE_SERIAL.println("client disconnected");
93 }
94 }

```

---

### 3 ESP is an Asynchronous HTTP server

Nous allons ici utiliser la bibliothèque "ESPAsyncWebServer" qui présente de nombreuses fonctionnalités intéressantes ;

<https://github.com/ESP32Async/ESPAsyncWebServer>

Construire un serveur web **asynchrone** présente plusieurs avantages, comme vous le pratiquez peut être en JS ? et comme mentionné dans la page GitHub de la bibliothèque.

Cela permet de,

- "Gérer plus d'une connexion en même temps" ;
- "Lorsque vous envoyez la réponse, vous êtes immédiatement prêt à gérer d'autres connexions pendant que le serveur se charge d'envoyer la réponse en arrière-plan" ;
- "Un moteur de traitement ... pour gérer les handlers" ;
- Et bien plus encore ...

Pour ceux, qui connaissent le package "express" en Javascript, ils retrouveront une similitude fonctionnelle.

- Pour ceux qui ne connaissent pas ... ils vont apprendre ;-)

**Il y a aussi des inconvénients** car l'"asynchrone" est quelque chose de complexe pour le programmeur, pour l'ESP et pour son OS (au niveau de la gestion des activités).

Il sera difficile d'utiliser plusieurs asynchronismes simultanément et d'avoir plus tard plusieurs serveurs asynchrones (HTTP, MQTT, ...) sur un même ESP.

La faute à l'OS ?, aux bibliothèques ?, à la programmation ?

- **Bref, si vous faites n'importe quoi ... Tout EXPLOSE ! :-)**

Donc on y va doucement et on lit bien les documentations pour être certain de la compatibilité des fonctions notamment dans la **gestion/utilisation** du `yield()` ...

- **On teste régulièrement !!!!!!!!!!!!!!!**

#### 3.1 Installation

Pour installer les deux bibliothèques nécessaires (AsyncTCP et ESPAsyncWebServer), j'ai juste extrait les archives obtenues par Git dans le répertoire "HomeRepertoire/Arduino/libraries".

- Il faut ensuite aussi re-ouvrir l'IDE Arduino.

## 3.2 Pour faire quoi?

L'ESP assure son rôle de serveur et répond selon l'URL/route qui lui est soumise.

Selon l'URL qui lui est soumise, il renvoie :

- soit une page HTML qui donne son statut,
- soit la valeur (String) d'un de ses capteurs,
- soit ...

C'est vous (et moi) qui allez définir l'API HTTP.

## 3.3 La partie Html

Dans l'exercice précédent, l'ESP (en tant que serveur) renvoyait une page HTML "ultra simple".

On pourrait continuer à placer du HTML dans le fichier ".ino" en utilisant une chaîne de caractère (PROGMEM, ...) mais ce n'est pas l'idéal

- ni à écrire (une grande chaîne de caractères)
- ni pour structurer le programme (mélange de Html et de C++).

<https://github.com/me-no-dev/ESPAsyncWebServer#send-large-webpage-from-progmem-containing-templates>

**On peut faire bien mieux** (notamment grâce à ESPAsyncWebServer) et pour cela on crée dans un répertoire "data"

1. un fichier HTML "data/index.html"
2. et un fichier CSS "data/esp.css"

Ce répertoire sera uploadé dans la "flash memory" de l'ESP grâce à SPIFFS **avant l'exécution du firmware.**

Une ébauche de fichier "data/index.html" vous est proposé :

```
1
2 <!DOCTYPE HTML><html>
3   <head>
4     <meta name="viewport" content="width=device-width, initial-scale=1">
5     <meta content="text/html; charset=utf-8" http-equiv="Content-Type">
6     <link rel="stylesheet"
7         href="https://use.fontawesome.com/releases/v5.7.2/css/all.css"
8         integrity="sha384-fnmOCqbTlWIlj8LyTjo7mOUStjsKC4pOpQbqyi7RrhN7udi9RwhKkMHpvLbHG9Sr"
9         crossorigin="anonymous">
10    <link rel="stylesheet" type="text/css" href="esp.css">
11
12    <title>ESP32</title>
13  </head>
14
15  <body>
16    <h1>ESP32</h1>
17    <h3>Object status :</h3>
18    Uptime      : %UPTIME% s<br/>
```

```

19      Where      : %WHERE% s<br/>
20
21      <h3>Network link status :</h3>
22      WiFi SSID  : %SSID%<br/>
23      MAC address : %MAC%<br/>
24      IP address  : %IP%<br/>
25
26      <h3>Sensors status :</h3>
27      Temperature : %TEMPERATURE% C<br/>
28      Light       : %LIGHT% Lumen<br/>
29      Cooler      : %COOLER%<br/>
30      Heater      : %HEATER%<br/>
31
32      <h3>Sensors status WITH GLYPHS :</h3>
33      <div class="grid-container">
34          <div class="grid-item">
35              <i class="fas fa-thermometer-half" style="color:#059e8a;"></i>
36          </div>
37          <div class="grid-item">
38              <span class="sensors-labels">Temperature</span>
39              <span id="temperature">%TEMPERATURE%</span>
40              <sup class="units">&deg;C</sup>
41          </div>
42          <div class="grid-item">
43              <i class="far fa-lightbulb" style="color:#00add6;"></i>
44          </div>
45          <div class="grid-item">
46              <span class="sensors-labels">Light</span>
47              <span id="light">%LIGHT%</span>
48              <sup class="units">Lumen</sup>
49          </div>
50      </div>
51
52      <h3>Temp. Regul. Thresholds :</h3>
53      Low Temp  : %LT% C<br/>
54      High Temp : %HT% C<br/>
55
56      <h3> Periodic status report to :</h3>
57      <form action="/target" method="post">
58          <label for="ip">IP Address :</label>
59          <input type="text" name="ip" placeholder="%PRT_IP%"/><br/>
60          <label for="port">Port :</label>
61          <input type="text" name="port" placeholder="%PRT_PORT%"/><br/>
62          <label for="sp"> Sampling Period (in seconds) :</label>
63          <input type="text" name="sp" placeholder="%PRT_T%"/><br/>
64          <input type="submit" value="Change reporting host !"/>
65      </form>
66
67      <script>
68          // The last version should use fetch API instead of XMLHttpRequest()
69          // my proposal with help from
70          // https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Fetching_data
71          // https://rapidapi.com/guides/api-requests-intervals
72          // https://medium.com/codingurukul/fetch-api-a-modern-replacement-for-xmlhttprequest-c24ced9ff8d3
73
74          function startUpdate(sensor){
75              setInterval(function () {
76                  fetch(sensor)
77                      // fetch() returns a promise. When we have received a response from the server,
78                      // the promise's `then()` handler is called with the response.
79                      .then( function (response) {
80                          // Our handler throws an error if the request did not succeed.

```

```

81         if (!response.ok) {
82             throw new Error(`HTTP error: ${response.status}`);
83         }
84         // Otherwise (if the response succeeded), our handler fetches the response
85         // as text by calling response.text(), and immediately returns the promise
86         // returned by `response.text()`.
87         return response.text();
88     })
89     // When response.text() has succeeded, the `then()` handler is called with
90     // the text, and we copy it into the `temperature` id.
91     .then(function (text){
92         document.getElementById(sensor).innerHTML = text;
93     })
94     // Catch any errors that might happen, and display a message
95     // in the navigator console.
96     .catch(function (error){
97         console.log(error);
98     });
99     }, 2000);
100 }
101
102 document.addEventListener('DOMContentLoaded', function() {
103     startUpdate('temperature');
104     startUpdate('light');
105 });
106 </script>
107 <script>
108     // Deprecated approach =====
109     // setInterval(function ( ) { // ===== Pour obtenir la temperature
110     //     var xhr = new XMLHttpRequest(); // Constructor
111     //     // XMLHttpRequest changes between states as it progresses
112     //     xhr.onreadystatechange = function() { // Handler to track XMLHttpRequest object state
113     //         // DONE = 4; when request complete and return OK (200)
114     //         if (this.readyState == 4 && this.status == 200) {
115     //             var r = this.response //renvoie le texte reçu d'un serveur suite à l'envoi d'une requête.
116     //             //console.log(r);
117     //             // .innerHTML method is used to change the html contents of a DOM object :
118     //             // temperature declare dans le span id du grid item
119     //             document.getElementById("temperature").innerHTML = r;
120     //         }
121     //     };
122     //     xhr.open("GET", "/temperaturee", false);
123     //     // Contrary to its name, does not open the connection.
124     //     // It only configures the request, but the network activity only
125     //     // starts with the call of send.
126     //     // true => asynchrone open call.
127     //     xhr.send(); //This method opens the connection and sends the request to server.
128     // }, 1000 ) ;
129     // setInterval(function ( ) { // ===== Pour obtenir la lumiere
130     //     var xhr = new XMLHttpRequest();
131     //     xhr.onreadystatechange = function() {
132     //         if (this.readyState == 4 && this.status == 200) {
133     //             document.getElementById("light").innerHTML = this.responseText;
134     //         }
135     //     };
136     //     xhr.open("GET", "/light", false);
137     //     xhr.send();
138     // }, 1000 ) ;
139 </script>
140 </body>
141 </html>

```



### 3.3.1 Quelques commentaires !

La première chose est de bien comprendre que **ce fichier HTML sera la réponse de l'ESP à une requête par exemple émise depuis un navigateur.**

- Il va donc être interprété par le navigateur !

La partie HTML fait classiquement apparaître deux parties :

1. Le **"header"** avec les "incantations" et paramétrages :
  - "meta" (ici optimisé pour les mobiles),
  - "link" (pour charger les icons qui sont des ressources externes),
  - "style" (si on avait laissé le CSS dans le header ... ce qui n'est plus le cas !).
2. Le **"body"** avec les informations qui représenteront le statut de l'ESP.

Dans ce body, j'attire votre attention sur les points qui suivent.

### 3.3.2 PLACEHOLDERS

Les **"placeholders"** (%TEMPERATURE%, %LIGHT%, ...) présents dans le HTML sont **destinés à être remplacés** par leurs valeurs respectives définies (par l'ESP) au moment où le serveur de l'ESP traitera la requête.

C'est la fonction **"processor()"** transmise au serveur Web de l'ESP (cf server.on(...)) qui est en charge de cela.

```

=====*/
String processor(const String& var){
  /* Replaces "placeholder" in html file with sensors values */
  /* accessors functions get_... are in sensors.ino file */

  //Serial.println(var);
  if(var == "TEMPERATURE"){
    return last_temp; // Pour chaque PLACEHOLDER .. on dit la valeur qui le remplace
                      // dans le HTML

    ...
    ...
  }
}
/*=====*/
void setup_http_routes(AsyncWebServer* server) {
  /*
   * Sets up AsyncWebServer and routes
   */
  ...
  // Declaring root handler, and action (given as a lambda with "request" parameter)
  // to be taken when root is requested
  auto root_handler = server->on("/", HTTP_GET, [] (AsyncWebServerRequest *request){

```

```
// https://forum.arduino.cc/t/please-explain-syntax-asyncwebserverrequest-request/1020064/2
USE_SERIAL.printf("Root route requested !\n");
/* This handler will download index.html (stored as LittleFS file) and will send it back */
request->send(LittleFS, "/index.html", String(), false, processor);
...
/*=====*/
```

Cette fonction que vous allez définir dans le ".ino" permettra ainsi de faire l'association entre "identificateur de placeholder" et "valeur du capteur correspondant".

### 3.3.3 Grid HTML

Une grid avec des glyphs ... pour jouer avec HTML ...

Vous pouvez trouver d'autres icones sur le site fontawesome :

<https://fontawesome.com/icons?d=gallery>

Le tag <i> fait le reste : <https://www.w3schools.com/icons/>

Dans cette grid, les valeurs sont périodiquement mises à jour par le script JS qui suit.

- C'est grâce à l'"id" de l'item de la grid que le script sait sur quoi il doit agir.

## 3.4 Le Script JS

Le "script" JS contenu dans la page permet d'installer des requêtes HTTP **périodiques coté client/navigateur**.

- La page affichée sur le navigateur va ainsi solliciter périodiquement le serveur (sur l'ESP) pour mettre à jour la température et la lumière.

J'ai essayé de "moderniser" le code en utilisant l'API Fetch ... j'espère que cela sera aussi solide.

## 3.5 La partie "Periodic status report to" (début du TODO)

L'idée de cette partie est de pouvoir configurer au niveau de l'ESP l'adresse d'un serveur de "reporting" auquel l'ESP enverrait périodiquement les valeurs de ses capteurs par des requêtes POST.

Grâce à ce formulaire HTML, l'ESP va recevoir une requête POST sur l'url "/target" dans laquelle les coordonnées de ce serveur seront indiquées.

- C'est à l'utilisateur devant le navigateur de remplir les champs et de cliquer !
- C'est à l'ESP de gérer cette requête et d'inscrire ce "reporting" et ensuite de le réaliser.

**Plus loin dans le TODO, ce serveur de reporting sera implémenté par un serveur/dashboard NodeRed.**

## 3.6 Remarque "debugging"

Vous allez sans doute (j'espère ?) faire évoluer ce code pour "coller" à l'exercice qui vous sera soumis.

- **On n'oublie pas la console du navigateur pour mettre au point et chercher les bugs sur les requêtes/réponses.**
- et puis il y a aussi cURL ...

### 3.7 La partie ESP

**ESPAsyncWebServer** est donc une bibliothèque permettant d'utiliser l'ESP comme un serveur Web Asynchrone.

La notion d'asynchronisme induit la possibilité d'associer des handlers qui sont utilisés pour exécuter (de façon asynchrone) des actions spécifiques selon la requête :

<https://github.com/ESP32Async/ESPAsyncWebServer/wiki#handlers-and-how-do-they-work>

On analyse quelques spécificités de ce code. On commence par le "setup" et la "loop" :

```
1  /*
2  * Auteur : G.Menez
3  * Fichier : http_as_serverasync_littlefs/http_as_serverasync_littlefs.ino
4
5  * Many sources :
6  => https://raw.githubusercontent.com/RuiSantosdoime/ESP32-Course/master/code/WiFi_Web_Server_DHT/WiFi_Web_Server_DHT.ino
7  => https://randomerdutorials.com/esp32-dht11-dht22-temperature-humidity-web-server-arduino-ide/
8  => Kevin Levy
9  */
10 /*===== Import required libraries =====*/
11 #include <ArduinoOTA.h>
12 #include "ArduinoJson.h"
13 #include <WiFi.h>
14 #include "AsyncTCP.h"
15 #include "ESPAsyncWebServer.h"
16 #include "wifi_utils.h"
17 #include "sensors.h"
18 #include "OneWire.h"
19 #include "DallasTemperature.h"
20 #include "FS.h"
21 #include <LittleFS.h>
22
23 #include "routes.h"
24
25 #define USE_SERIAL Serial
26
27 void setup_OTA(); // from ota.ino
28
29 /*===== ESP GPIO configuration =====*/
30 /* --- LED ---*/
31 const int LEDpin = 19; // LED will use GPIO pin 19
32 /* --- Light ---*/
33 const int LightPin = A5; // Read analog input on ADC1_CHANNEL_5 (GPIO 33)
34 /* --- Temperature ---*/
35 OneWire oneWire(23); // Pour utiliser une entite oneWire sur le port 23
36 DallasTemperature TempSensor(&oneWire); // Cette entite est utilisee par le capteur de temperature
37
38 /*===== ESP Statut =====*/
39 // Ces variables permettent d'avoir une representation
40 // interne au programme du statut "electrique" de l'objet.
41 // Car on ne peut pas "interroger" une GPIO pour lui demander !
42 String LEDState = "off";
43 String last_temp;
44 String last_light;
45
46 /*===== Process configuration =====*/
47 // Set timer
48 unsigned long loop_period = 10L * 1000; /* => 10000ms : 10 s */
49
50 // Create AsyncWebServer object on port 80
51 AsyncWebServer server(80);
```

```

52
53 // Thresholds
54 short int Light_threshold = 250; // Less => night, more => day
55
56 // Host for periodic data report
57 String target_ip = "";
58 int target_port = 1880;
59 int target_sp = 0; // Remaining time before the ESP stops transmitting
60
61 /*=====*/
62 /* Delay can be non compatible with OTA handle or with async !!!!
63  * because ESP32 pauses your program during the delay().
64  * If next OTA request is generated while Arduino is paused waiting for
65  * the delay() to pass, your program will miss that request.
66  * => Use a non-blocking method, using millis() and a delta to decide to read or not. eg */
67 void DoSmtg(int delai){
68     static uint32_t tick = 0;
69     if ( millis() - tick < delai) {
70         return;
71     }
72     else { /* Do stuff here every 10 second */
73         USE_SERIAL.print("deja 10 secondes écoulées ! \n");
74         tick = millis();
75
76         wifi_printstatus(0);
77
78         /*last_temp = atoi(get_temperature(TempSensor).c_str());
79         last_temp = get_temperature(TempSensor);
80         last_light = get_light(LightPin);
81
82         if (target_sp>0){
83             Serial.println("Je dois envoyer une requete POST là :)");
84             Serial.println(target_ip);
85             Serial.println(target_port);
86             Serial.println(target_sp);
87         }
88
89     }
90 }
91
92 /*=====*/
93 /*---- Arduino IDE paradigm : setup+loop ----*/
94 /*=====*/
95
96 void setup(){
97     /* Serial connection -----*/
98     USE_SERIAL.begin(9600);
99     while(!USE_SERIAL); //wait for a serial connection
100
101     /* WiFi connection -----*/
102     String hostname = "Mon petit objet ESP32";
103     wifi_connect_multi(hostname);
104
105     /* WiFi status -----*/
106     if (WiFi.status() == WL_CONNECTED){
107         USE_SERIAL.print("\nWiFi connected : yes ! \n");
108         wifi_printstatus(0);
109     }
110     else {
111         USE_SERIAL.print("\nWiFi connected : no ! \n");
112         // ESP.restart();
113     }
114 }
115
116 // Init OTA
117 //setup_OTA();
118

```

```

119 // Initialize the LED
120 setup_led(LEDpin, OUTPUT, LOW);
121
122 // Init temperature sensor
123 TempSensor.begin();
124
125 // Initialize LittleFS
126 if(!LittleFS.begin(true)){
127     Serial.println("An Error has occurred while mounting LittleFS");
128     return;
129 }
130
131 // Setup routes of the ESP Web server
132 setup_http_routes(&server);
133
134 // Start ESP Web server
135 server.begin();
136 }
137
138
139 void loop(){
140     int delai = 10000;
141     /* OTA*/
142     //ArduinoOTA.handle();
143
144     DoSmtg(delai);
145
146     //delay(loop_period); // ms
147 }

```

---

### 3.7.1 Le setup et la loop

La fonction "setup" initialise pas mal de choses :

- les capteurs,
- le SPIFFS qui va permettre d'y placer le HTML et le CSS,
- les routes sur serveur HTTP,

puis elle démarre le serveur HTTP.

La "loop()" évite d'utiliser la fonction `delay()` qui n'est pas compatible avec l'asynchronisme ambiant.

La boucle met à jour les valeurs des capteurs qui participent au status de l'objet.

- Il ne faudrait surtout pas que la requête accède directement au capteur.

Lire physiquement une température peut prendre jusqu'à 1 seconde .

### 3.7.2 Les routes et les Handlers

Les requêtes à venir et auxquelles on souhaite répondre sont caractérisées

- par un type,
- une route et
- une lambda C++ qui prend en paramètre la requête et qui invoque une méthode (par exemple "send" ou "send\_P" ou ...)

<https://github.com/ESP32Async/ESPAsyncWebServer/wiki#request-life-cycle>

```
1  /*
2  * Auteur : G.Menez
3  * Fichier : http_as_serverasync_littlefs/routes.ino
4  */
5
6  #include "ESPAsyncWebServer.h"
7  #include "routes.h"
8  #include "FS.h"
9  #include <LittleFS.h>
10
11 #define USE_SERIAL Serial
12 extern String last_temp, last_light;
13
14 /*=====*/
15 String processor(const String & var){
16     /* Replaces "placeholder" in html file with sensors values */
17     /* accessors functions get_... are in sensors.ino file */
18
19     //Serial.println(var);
20     if(var == "TEMPERATURE"){
21         return last_temp; // Pour chaque PLACEHOLDER .. on dit la valeur qui le remplace
22                             // dans le HTML
23         /* On aimerait écrire : return get_temperature(TempSensor);
24          * mais c'est un exemple de ce qu'il ne faut surtout pas écrire ! */
25     }
26     else if(var == "LIGHT"){
27         return last_light;
28     }
29     return String(); // parce que !=> cf doc de asyncwebserver
30 }
31
32 /*=====*/
33 void setup_http_routes(AsyncWebServer* server) {
34     /*
35      * Sets up AsyncWebServer and routes
36      */
37
38     // La fonction serveStatic(); indique au serveur web d'envoyer les fichiers demandés
39     // par le navigateur (comme les images, les feuilles de style, les sources JavaScript, etc.)
40     // et qui sont stockés dans un système de fichiers
41     // https://forum.arduino.cc/t/cherche-renseignements-sur-serveur-servestatic/1182775
42     // => premier param la route et second param le repertoire servi (dans le LittleFS)
43     server->serveStatic("/", LittleFS, "/").setTemplateProcessor(processor);
44
45     // Declaring root handler, and action (given as a lambda with "request" parameter)
46     // to be taken when root is requested
47     auto root_handler = server->on("/", HTTP_GET, [](AsyncWebServerRequest *request){
48         // https://forum.arduino.cc/t/please-explain-syntax-asyncwebserverrequest-request/1020064/2
49         USE_SERIAL.printf("Root route requested !\n");
50         /* This handler will download index.html (stored as LittleFS file) and will send it back */
51         request->send(LittleFS, "/index.html", String(), false, processor);
52         // cf "Respond with content coming from a File containing templates" section in manual !
53         // https://github.com/me-no-dev/ESPAsyncWebServer
54         // request->send_P(200, "text/html", page_html, processor); // if page_html was a string .
55     });
56
57     #ifdef CSS
58     // Route to load style.css file
59     server.on("/esp.css", HTTP_GET, [](AsyncWebServerRequest *request){
60         request->send(LittleFS, "/esp.css", "text/css");
61     });
62     #endif
```

```

63
64 server->on("/temperature", HTTP_GET, [] (AsyncWebServerRequest *request){
65     /* The most simple route => hope a response with temperature value */
66     USE_SERIAL.printf("GET /temperature request \n");
67     /* Exemple de ce qu'il ne faut surtout pas écrire car yield + async => core dump ! */
68     //request->send_P(200, "text/plain", get_temperature(TempSensor).c_str());
69     request->send_P(200, "text/plain", last_temp.c_str());
70 });
71
72 server->on("/light", HTTP_GET, [] (AsyncWebServerRequest *request){
73     /* The most simple route => hope a response with light value */
74     request->send_P(200, "text/plain", last_light.c_str());
75 });
76
77 // This route allows users to change thresholds values through GET params
78 server->on("/set", HTTP_GET, [] (AsyncWebServerRequest *request){
79     /* A route with a side effect : this get request has a param and should
80      * set a new light_threshold ... used for regulation !
81      */
82     if (request->hasArg("light_threshold")) { // request may have arguments
83         Light_threshold = atoi(request->arg("light_threshold").c_str());
84         request->send_P(200, "text/plain", "Threshold Set !");
85     }
86 });
87
88 server->on("/target", HTTP_POST, [] (AsyncWebServerRequest *request){
89     /* A route receiving a POST request with Internet coordinates
90      * of the reporting target host.
91      */
92     Serial.println("Receive Request for a periodic report !");
93     if (request->hasArg("ip") &&
94         request->hasArg("port") &&
95         request->hasArg("sp")) {
96         target_ip = request->arg("ip");
97         target_port = atoi(request->arg("port").c_str());
98         target_sp = atoi(request->arg("sp").c_str());
99     }
100     request->send(LittleFS, "/index.html", String(), false, processor);
101 });
102
103 // If request doesn't match any route, returns 404.
104 server->onNotFound([] (AsyncWebServerRequest *request){
105     request->send(404);
106 });
107 }
108 /*=====*/

```

Ainsi, par exemple, du fait de la présence de l'appel :

---

```

// Declaring root handler, and action (given as a lambda with "request" parameter)
// to be taken when root is requested
auto root_handler = server->on("/", HTTP_GET, [] (AsyncWebServerRequest *request){
    // https://forum.arduino.cc/t/please-explain-syntax-asyncwebserverrequest-request/1020064/2
    USE_SERIAL.printf("Root route requested !\n");
    /* This handler will download index.html (stored as LittleFS file) and will send it back */
    request->send(LittleFS, "/index.html", String(), false, processor);
});

```

---

1. si l'ESP reçoit une requête GET avec le path "/" alors en tant que serveur il répond en envoyant le fichier "/index.html" contenu en SPIFFS.

2. De plus, **du fait de la présence de la fonction "processor", les placeholders seront substitués.**

Dans la figure qui suit, tous ne sont pas présents. Il vous reste du travail !



192.168.1.45

Débuter avec Firefox
 
 Les plus visités
 [sl Déposer une annonce](#)
 Débu

## ESP32

**Object status :**

Uptime : s  
Where : s

**Network link status :**

WiFi SSID :  
MAC address :  
IP address :

**Sensors status :**

Temperature : 25.31 C  
Light : 0 Lumen  
Cooler :  
Heater :

**Sensors status WITH GLYPHS :**

|  |                      |
|--|----------------------|
|  | Temperature 25.69 °C |
|  | Light 1070 Lumen     |

**Temp. Regul. Thresholds :**

Low Temp : C  
High Temp : C

**Periodic status report to :**

IP Address :

Port :

Sampling Period (in seconds) :

[Change reporting host !](#)

La bibliothèque permet aussi de définir des requêtes renvoyant tout simplement du texte ...si l'API de l'ESP le prévoit. La preuve :

```
~> curl http://192.168.219.4:80/temperature
26.12
```

### 3.8 Le delay, le yield et FreeRTOS

Le NodeMCU ESP32 est basé sur le microprocesseur dual-core Xtensa 32-bit LX6 qui embarque l'OS FreeRTOS.

- C'est un dual-core et donc le parallélisme est effectif !

Lorsqu'on utilise l'IDE Arduino, le programme généré s'exécute classiquement/sagement par défaut sur le cœur **"Application CPU"** (APP\_CPU).

Le **second cœur "Protocol CPU"** (PRO\_CPU) s'occupe de WiFi, Bluetooth, de la pile TCP/IP, et de tous les autres périphériques internes : SPI, I2C, ADC etc.

FreeRTOS, comme tout "Operating System", se voit confier l'ordonnancement des tâches.

- <https://www.freertos.org/a00020.html>
- <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos.html>

Le programmeur peut intervenir "plus ou moins" sur l'organisation "par défaut" dans le but de rechercher et d'exploiter (sans doute) le parallélisme potentiel de son application.

Il va alors

- définir des tasks,
- placer ces tasks sur un core,
- ...

Le risque majeur d'une application parallèle, c'est le "blocage" d'une ressource (un cœur) par une tâche empêchant de fait d'y placer d'autres tâches mais aussi perturbant l'algorithme de scheduling/ordonnancement de l'OS.

La fonction `delay()`, que l'on a déjà utilisée, pourrait (mais on va voir que ce n'est pas le cas) faire partie des fonctions "bloquantes" puisqu'elle bloque la tâche qui l'appelle pendant un certain temps. Plus précisément (que l'IDE Arduino), `delay()` appelle `vTaskDelay()`. `vTaskDelay()` n'est pas bloquant, sauf pour la tâche qui l'appelle.

- Une autre tâche peut donc prendre la main pendant les appels à `delay()`.

Une fonction **qui ne veut pas devenir bloquante peut faire appel à la fonction `yield()` pour informer l'OS** qu'il peut effectuer un changement de tâche :

- "taskYIELD() is used to request a context switch to another task.  
However, if there are no other tasks at a higher or equal priority to the task that calls taskYIELD() then the RTOS scheduler will simply select the task that called taskYIELD() to run again."

```

1  void loop() {
2      // put your main code here, to run repeatedly:
3      while (1){ // Boucle infinie d'attente
4          //yield(); // Avec le yield on redonne la main à l'OS à chaque boucle!
5          Serial.println("Waiting");
6      }
7  }

```

Tous les objets/microcontrôleurs n'implémentent pas forcément la fonction `yield()`. L'ESP qui est "dual core implémente" réellement cette fonction qui doit être appelée par tout code bloquant.

Ainsi `yield()` appelle `vTaskSwitchContext()` qui passe éventuellement la main à une autre tâche.

- Cela permet par exemple dans une implémentation du type `WebServer` de ne pas bloquer la tâche WIFI pendant les appels de `handleClient()`, qui est appelée en boucle par `loop()`, ce qui est théoriquement bloquant.

Si `handleClient()` a une requête à traiter, elle la traite, sinon elle appelle `yield()`. Si `yield()` n'était pas appelée, la tâche WIFI n'aurait jamais la main, et `handleClient()` ne serait jamais alimenté.

---

D'où vient le crash, si on appelle `get_temperature()` dans le service ?

[https://www.reddit.com/r/esp8266/comments/iofuxy/esp8266\\_crashes\\_anytime\\_i\\_connect\\_to\\_the\\_ip/](https://www.reddit.com/r/esp8266/comments/iofuxy/esp8266_crashes_anytime_i_connect_to_the_ip/)

"You're not explicitly calling ``yield()`` but it turns out that the Dallas Temperature library does call it when you take a reading.

You're using `ESPAsyncWebServer`. Its documentation explicitly says that you must not call ``yield()`` or ``delay()`` while handling a callback (under Important things to remember):

You can not use `yield` or `delay` or any function that uses them inside the callbacks

This is because `ESPAsyncWebServer` uses `ESPAsyncTCP`, which calls TCP-related callbacks from its own separate task (provided by the underlying SDK that the Arduino Core for ESP8266 is built on). Calling ``delay()`` or ``yield()`` in a callback in the context of that task can break assumptions about how tasks are run and cause crashes like you're seeing.

That means you cannot safely call the `DallasTemperature` methods ``getTempFByIndex()`` or ``getTempCByIndex()`` from within an async web server callback.

Instead you'll need to restructure your code so that in ``loop()`` you read the temperatures periodically, store them in global variables and then return the values of those variables in the callbacks rather than calling the Dallas Temperature library directly."

## 4 TODO : The final step of this lab !

Le contexte de ce travail est celui du dialogue en "HTTP" entre un objet (ESP) et une station que l'on pourrait qualifier de "supervision".

- En "pré-requis", on fait l'hypothèse que vous êtes montés en compétences en étudiant les exemples qui précèdent.

Il y a des codes à lire et à comprendre et il y a aussi les outils à essayer. Je les énumère pour ne pas oublier :

1. HTTP
2. Json
3. SPIFFS / LittleFS
4. Node Red
5. OTA ?

L'idée globale de ce qui suit est de démontrer une maîtrise des échanges d'informations entre les composants de l'écosystème IoT que vous êtes en train de créer.

### 4.1 Cahier des charges de ce qui va être évalué !

Votre travail fait intervenir 3 éléments dans le contexte de la gestion de régulation de température du step 1.0 :

1. **L'ESP en tant qu'"objet"**,
2. **Une page standard dans un navigateur** qui jouera le rôle d'une "machine" de configuration/d'administration.  
Un peu comme la page qui permet de configurer votre box (ADSL ou fibre). C'est sur cette page que le status de l'ESP sera affiché.
3. **Un flot Node-Red dans un navigateur** qui jouera le rôle du serveur de données au centre du réseau.
  - C'est le serveur de "reporting" (périodique) ciblé par l'ESP.

En tant que tel, il va proposer une visualisation des données/status de l'ESP et du processus de régulation de façon plus générale :

- Il propose donc un dashboard/tableau de bord ... sans doute le même que celui du travail au step 1.5 ? Seule différence, le noeud de réception qui sera un "httpin" !

Pour l'instant, ce composant n'implémente pas de persistance car il n'utilise pas de bases de données.

L'ESP met à disposition de l'écosystème une API qui doit savoir répondre à plusieurs requêtes que vous allez construire.

#### 4.1.1 Configuration/administration de l'ESP

Premier élément de l'API, une requête GET avec le path "/" doit permettre d'obtenir la page d'administration de l'ESP.

La réponse est une page HTML qui présente un statut de l'ESP :

- Identité de l'ESP,
- Coordonnées physiques,
- Coordonnées de connexion,
- Valeurs de ses capteurs,
- Valeurs de ses seuils, ...

Par rapport à ce que je donne dans le fichier "data/index.html", il faut finir la gestion des "placeholders" pour comprendre comment cela fonctionne.

- J'insiste sur l'utilisation des "placeholders" car ils sont bien plus économes (en réseau et en CPU) qu'une potentielle approche AJAX.
- Par contre, ils sont statiques ... MAIS cette page "index.html" n'est pas destinée à remplacer le dashboard nodered.

Si vous êtes à l'aise avec HTML et CSS alors vous pouvez rendre les choses un peu plus "jolies" et enrichir la page:

- Faites propre et simple (car ce n'est pas un cours sur Html !)
- Il faut que la page donne "toutes" les informations possibles.

Imaginez une page de configuration de "Box" ... mais **NE FAITES PAS TROP** compliqué ; **on garde les solutions plus dynamiques (Javascript avancé) ou plus complexes (insertion d'une database) pour plus tard.**

#### 4.1.2 Reporting de l'ESP

Dans la page d'administration de l'ESP, **il y a un obligatoirement un formulaire** qui permet d'informer l'ESP que l'on souhaite activer le reporting.

Ce formulaire envoie une requête POST avec le path "/target" et avec des paramètres qui désigneront la station "serveur de données" :

- son "ip"
- son "port"
- et "sp" : la période d'échantillonnage/reporting souhaitée.

Une valeur zéro **bloque** l'émission périodique.

A partir du moment où cette information est connue de l'ESP (et avec une valeur positive non nulle) ce dernier va émettre périodiquement ses données vers ce port **avec une requête POST** :

- avec **comme path**, à destination du serveur NodeRed, la chaîne "/esp",
- **comme paramètre** son adresse mac car le serveur de reporting pourrait être la "cible" de plusieurs objets et il pourra ainsi les identifier avant même de lire le body,
- **et en body** le schéma Json introduit dans un sujet précédent.

#### 4.1.3 "Get a value" de l'ESP

L'API de l'ESP contient aussi une requête GET avec le path `"/value"` plus des paramètres qui spécifient les noms des valeurs attendues.

Les valeurs demandées peuvent être choisies parmi la liste :

- temperature
- light
- fire
- ht
- lt

Une telle requête peut donc retourner plusieurs informations ... que vous exprimerez en JSON !

Par exemple :

```
http://192.168.1.100/value?temperature&light
```

renverra

```
{"temperature":23.4,"light":843}
```

Si un paramètre n'existe pas, la réponse pourrait être une réponse "404".

#### 4.1.4 "Set a value" de l'ESP

Une requête GET avec le path `"/set"` et en paramètres les noms/valeurs permettant de positionner les paramètres de régulation de l'objet.

Par exemple :

- `http://192.168.1.1/set?cool=on` pour allumer le "cooler" de l'ESP .

Si un paramètre n'existe pas, la réponse pourrait être une réponse "404".

## 4.2 Dashboard

Pour finir le TODO, vous devez faire évoluer le dashboard Node-RED de la session précédente **pour venir l'interfacer avec la requête POST** de statut périodique mise en place à l'exercice précédent.

- Cette requête contient le JSON du status de l'objet déjà développé!

Vous vérifierez dans la fenêtre debug de NodeRed que le JSON reçu est conforme à ce qu'il devrait être.

Pour réaliser cet interface, vous utilisez les noeuds "http in" et "http response" dans la catégorie "network".

### 4.2.1 Pour tester votre dashboard

Pour simuler la présence de l'objet, vous pourrez utiliser des requêtes cURL :

```
curl -v -d '{"temperature":23, "light":1234}' -H "Content-type: application/json" \
http://machinetarget:port/esp?mac="38:14:28:db:56:9d"
```

La structure JSON utilisé dans cet exemple n'est pas celle qui est attendue mais c'est un exemple simple.

Pour tester votre dashboard, vous choisissez l'adresse du NodeRed au niveau du formulaire de configuration de l'ESP

## 5 Annexes

Je vous donne quelques liens qui peuvent aider dans la compréhension et la réalisation de votre travail.  
C'est un peu en vrac mais au niveau Master vous devez être capable de remettre de l'ordre ;-)

### 5.1 Quelques liens pour aider ...

- Formulation requête POST "à la main" : <https://davidgs.com/fr/posts/category/iot/iot-software/using-tls-to-post-from-esp32/>
- <https://stackoverflow.com/questions/20906066/making-href-anchor-tag-request-post-instead-of-get>
- <https://stackoverflow.com/questions/3915917/make-a-link-use-post-instead-of-get>
- Plutôt orienté Javascript : <https://stackoverflow.com/questions/426310/how-do-you-post-data-with-a-link>
- <https://github.com/me-no-dev/ESPAsyncWebServer#get-post-and-file-parameters>
- <https://github.com/esp8266/Arduino/issues/1390>
- <https://randomnerdtutorials.com/esp32-http-get-post-arduino/>
- [https://github.com/RuiSantosdotme/Random-Nerd-Tutorials/blob/master/Projects/ESP32/HTTP/ESP32\\_HTTP\\_POST.ino](https://github.com/RuiSantosdotme/Random-Nerd-Tutorials/blob/master/Projects/ESP32/HTTP/ESP32_HTTP_POST.ino)
- <https://techtutorialsx.com/2017/05/20/esp32-http-post-requests/>
- <https://randomnerdtutorials.com/esp8266-nodemcu-http-post-ifttt-thingspeak-arduino/>

### 5.2 Exemple de formulaire en HTML

Le code source de <http://httpbin.org/forms/post> fournit un exemple HTML d'utilisation d'une requête POST à partir de la balise <form ... :

```
1      <!DOCTYPE html>
2      <html>
3      <head>
4      </head>
5      <body>
6
7      <!-- Example form from HTML5 spec http://www.w3.org/TR/html5/forms.html#writing-a-form's-user-interface -->
8
9      <form method="post" action="/post">
10     <p><label>Customer name: <input name="custname"></label></p>
11     <p><label>Telephone: <input type="tel" name="custtel"></label></p>
12     <p><label>E-mail address: <input type="email" name="custemail"></label></p>
13     <fieldset>
14     <legend> Pizza Size </legend>
15     <p><label> <input type="radio" name="size" value="small"> Small </label></p>
16     <p><label> <input type="radio" name="size" value="medium"> Medium </label></p>
17     <p><label> <input type="radio" name="size" value="large"> Large </label></p>
18     </fieldset>
19     <fieldset>
20     <legend> Pizza Toppings </legend>
21     <p><label> <input type="checkbox" name="topping" value="bacon"> Bacon </label></p>
22     <p><label> <input type="checkbox" name="topping" value="cheese"> Extra Cheese </label></p>
23     <p><label> <input type="checkbox" name="topping" value="onion"> Onion </label></p>
24     <p><label> <input type="checkbox" name="topping" value="mushroom"> Mushroom </label></p>
25     </fieldset>
26     <p><label>Preferred delivery time: <input type="time" min="11:00" max="21:00" step="900" name="delivery"></label></p>
27     <p><label>Delivery instructions: <textarea name="comments"></textarea></label></p>
28     <p><button>Submit order</button></p>
29     </form>
30
31     </body>
32     </html>
```